PRYZM RESEARCH • WHITEPAPER V1.0

Pricing Recommendation Methodology

An expected-margin optimiser for industrial manufacturers, with forecast-aware demand, customer-level churn response, and Monte-Carlo uncertainty.

Audience. Investors, prospective customers, and partners evaluating Pryzm's pricing intelligence stack.

Scope. The mathematical foundation of Pryzm's price-recommendation engine, the forecasting backbone it consumes, the simulation modules used to certify decisions before rollout, and the validation methodology that quantifies how often the engine is right.

Living document. Numbers, models, and weights are versioned. This file is updated as backtests refresh and as additional data sources come online.

Abstract

We describe the mathematical and software architecture of Pryzm's price-recommendation engine. The engine optimises an explicit, financially interpretable objective — expected annual contribution margin across each SKU's eligible customer book — subject to cost, market, contract, and churn-rate constraints. Demand and cost vintages are sourced from an ensembled time-series forecasting backbone whose 12-month walk-forward MAPE on a manufacturer's 2025 revenue book is 13.9%. Customer-level churn response is modelled per cohort, expected loss is computed from a 24-month discounted lifetime-value (LTV) horizon, and the full input distribution is propagated to the score through Monte Carlo simulation so that every recommendation carries an explicit confidence band. We validate against the 2025 invoice book in a strict walk-forward protocol and report the resulting accuracy metrics. We also describe the simulation module — including a Monte-Carlo what-if engine and a Thompson-sampling exploration policy slated for the post-AB-test phase — and the wiring path from the validated notebook into the production FastAPI service. The system's design follows recent literature on Bayesian LTV-aware dynamic pricing and constrained churn-bounded optimisation, with explicit numerical reproducibility throughout.

Keywords. B2B price optimisation, contribution-margin maximisation, Bayesian win-probability, churn-aware pricing, Monte Carlo simulation, Chronos, AutoETS, MinT reconciliation, Thompson sampling.

Living document

This methodology evolves. The change log in Appendix B records every revision to models, weights, accuracy figures, and constraint defaults. Pryzm maintains a single source of truth: `docs/whitepaper/pryzm_pricing_methodology.tex`, versioned in the production repository.

Contents

1 Introduction

1.1 The problem

Industrial manufacturers price thousands of SKUs against heterogeneous customer relationships. Catalogue prices drift away from cost; contractual prices are renewed annually without measuring elasticity; quote-stage discounts are granted without measuring win probability; churn risk is observed only in retrospect. The result is a portfolio that is simultaneously over-discounted on inelastic accounts and over-priced on price-sensitive ones, with margin leakage that no single dashboard surfaces.

A recommendation engine must answer four questions for every price it proposes:

1. Which price maximises *expected annual contribution* given current cost, demand, and competitive signal?
2. How confident is the recommendation, and what is the distribution of outcomes?
3. How much margin is lost if the customer churns at the proposed price, and is that loss exceeded by the recovery on retained customers?
4. What is the breakeven price, and how sensitive is the recommendation to each input?

1.2 Design principles

Pryzm's engine is built to be **explicit, auditable, and forecast-aware**.

- *Explicit objective.* The scoring function is a single closed-form expression of expected margin; there is no opaque end-to-end neural pricer.
- *Decomposable inputs.* Each term — win probability, retention, demand, cost, LTV — is sourced from a separately validated model whose accuracy is published.
- *Constrained.* Recommendations live inside a cost floor, a market ceiling, a contractual delta cap, and a portfolio churn cap. No recommendation can violate guardrails silently.
- *Uncertainty-first.* Monte Carlo propagates input distributions to the score; the engine returns a 90% confidence interval, not a point.
- *Backtested.* Every model version is gated on a walk-forward backtest against the prior calendar year. Accuracy is published.

> **Key takeaway.** Pryzm does not predict "the right price". It computes the distribution of contribution margin as a function of price, identifies the maximiser, and reports the breakeven point and the confidence band — so that the pricing analyst inherits a quantified decision, not a black-box answer.

2 Methodology overview

The engine consumes seven inputs per (SKU, customer, candidate price) tuple and returns a scored recommendation per SKU. Figure 1 shows the high-level data flow.

2.1 Why not an end-to-end neural pricer?

End-to-end neural models can outperform decomposed pipelines on dense consumer data where billions of transactions per category are available. In an industrial B2B portfolio the sample size per (SKU, customer-segment) cell is typically in the low hundreds and the cost of a single bad recommendation is large. The literature on B2B dynamic pricing converges on the same prescription: *treat LTV as a forecast with explicit uncertainty, optimise expected contribution subject to a churn cap, and never overreact to noise* [1, 3, 2]. A decomposed, interpretable engine is the only design that yields auditable reasoning, which is itself a hard requirement for our buyer.

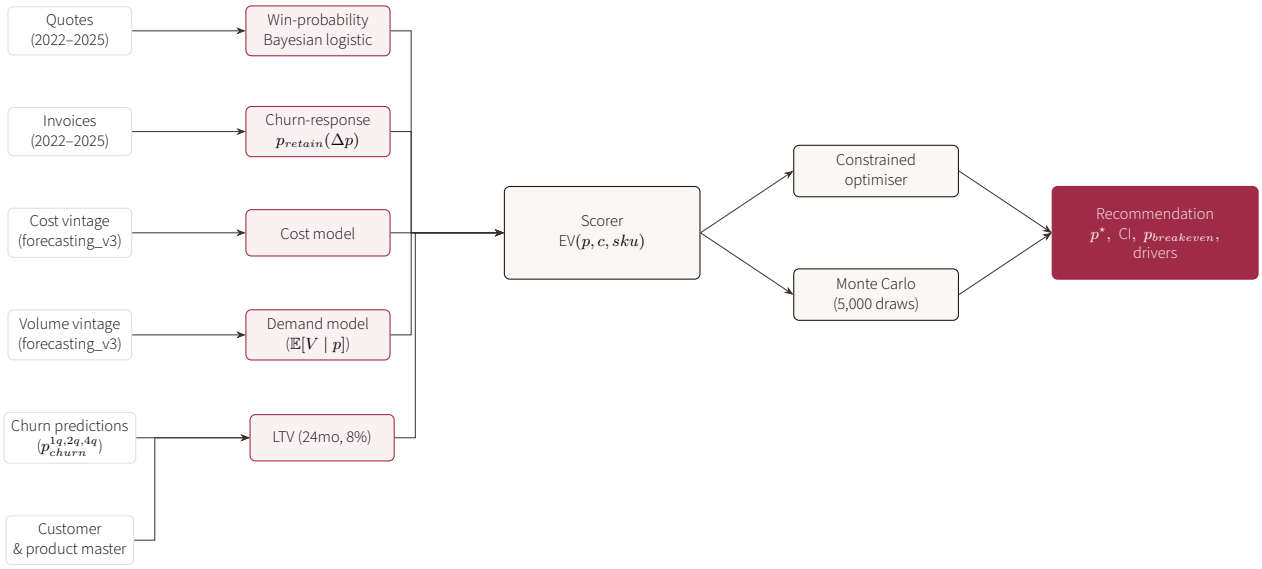

Figure 1: End-to-end data flow. Five families of input models (rose) feed a single scorer (dark grey) that is then queried by both a constrained optimiser (for the point recommendation) and a Monte-Carlo simulator (for the confidence band). The final recommendation packet contains the optimal price $p^{\star}$, a 90% confidence interval, the breakeven price, and per-driver attribution.

3 Mathematical formulation

3.1 Per-customer expected value

For a single SKU, a candidate price p, and an eligible customer c, the expected annual contribution is:

$$\begin{aligned}\mathbb{E}[\text{margin} \mid p, c] \;=\; & \underbrace{P_{\text{win}}(p \mid \mathbf{sku}, \text{seg}(c)) \cdot P_{\text{retain}}(p \mid c)}_{\text{retain-weighted}} \cdot \mathbb{E}[V_{12} \mid p, \mathbf{sku}, c] \cdot \big(p - k_{12}(\mathbf{sku})\big) \\ & - \underbrace{P_{\text{churn}}(p \mid c) \cdot L(c, \mathbf{sku})}_{\text{churn loss}}\end{aligned} \tag{1}$$

where

- $P_{\text{win}}(p \mid \mathbf{sku}, \text{seg}(c))$ is the modelled probability that the customer accepts a quote at price p, conditional on the SKU and the customer segment (see §5),
- $P_{\text{retain}}(p \mid c) = 1 - P_{\text{churn}}(p \mid c)$ is the customer-level retention probability (see §6),
- $\mathbb{E}[V_{12} \mid p, \mathbf{sku}, c]$ is the expected 12-month volume of this customer on this SKU conditioned on price (see §7),
- $k_{12}(\mathbf{sku})$ is the 12-month forecast unit cost (see §8),
- $L(c, \mathbf{sku})$ is the discounted 24-month lost contribution if the customer churns (see §9).

3.2 Cluster aggregation and optimisation

The cluster-level score is the sum across the SKU's eligible customer book $\mathcal{C}$:

$$S(p) \;=\; \sum_{c \in \mathcal{C}} \mathbb{E}[\text{margin} \mid p, c]. \tag{2}$$

The recommended price is the constrained maximiser:

$$p^\star \;=\; \arg\max_{p \in \mathcal{P}} \; S(p) \quad \text{s.t.} \quad \begin{cases} p \geq k_{12}(\text{sku}) + \tau_k & \text{(cost floor with safety margin } \tau_k\text{)} \\ p \leq m(\text{sku}) & \text{(market ceiling when known)} \\ \left|p - p_{\text{cur}}\right| / p_{\text{cur}} \leq \Delta_{\max} & \text{(contractual cap, default } 10\%\text{)} \\ \frac{1}{|\mathcal{C}|} \sum_c P_{\text{churn}}(p \mid c) - P_{\text{churn}}(p_{\text{cur}} \mid c) \leq \kappa & \text{(churn cap, default } 2 \text{ pp)} \end{cases} \tag{3}$$

3.3 Breakeven and sensitivity

The breakeven price is defined as the minimum price at which the score is non-negative:

$$p_{\text{breakeven}} \;=\; \min\{p \in \mathcal{P} \;:\; S(p) \geq 0\}. \tag{4}$$

The relative sensitivity to each input x_i at $p^\star$ is reported via marginal removal (a SHAP-style leave-one-out attribution):

$$\phi_i \;=\; S(p^\star) - S^{(-i)}(p^\star), \tag{5}$$

where $S^{(-i)}(p)$ is the score recomputed with input i replaced by its uninformative prior.

> **Key takeaway.** The objective is one expression. Every term is the output of a model whose accuracy we report. There is no hidden weight, no black box, and no recommendation without a corresponding $S(p)$, confidence band, and breakeven.

4 Forecasting backbone

The engine consumes monthly volume and unit-cost forecasts at a per-SKU resolution. These are produced by Pryzm's forecasting v3 pipeline, which is itself a bake-off of statistical, gradient-boosted, and pretrained transformer models, reconciled with a Minimum Trace (MinT-OLS) reconciler across the SKU hierarchy.

4.1 Models in the bake-off

- **Statistical baselines.** Seasonal-Naive(s=12), Theta, AutoETS.
- **Gradient-boosted.** LightGBM with calendar and lag features.
- **Foundation models.** Amazon *Chronos-Bolt-base* (univariate) and *Chronos-Bolt-base + FRED macro covariates* via AutoGluon-TimeSeries, with WPU101, copper, aluminium, Brent crude, EUR/USD, energy index, German 10y yield, and industrial production as exogenous regressors.
- **Tiny Time Mixer (TTM-r2).** A pretrained multivariate transformer from IBM Research.
- **Ensemble.** Equal-weighted median of Theta, AutoETS, and Seasonal-Naive — our current production winner on revenue.

4.2 2025 walk-forward results

We train every candidate on 2022-01 through 2024-12 and forecast Jan–Dec 2025. The forecast is compared against the actual 2025 invoice and cost realisations. Table 1 summarises the leaderboard.

4.3 Why these accuracy metrics

- **MAPE** is the natural metric for a pricing context because the downstream economic loss scales with relative error in revenue or cost, not absolute error.

Table 1: 2025 walk-forward backtest. Train 2022–2024, forecast 12 months 2025. MAPE on monthly point forecasts; bias is signed total-year error.

Model	MAPE %	Bias %	RMSE
Revenue (Σ_{2025} = €7.37M actual)			
Ensemble [Theta, ETS, SN]	**13.86**	−14.98	112,940
AutoETS	14.27	−14.54	116,646
AutoETS + MinT-OLS reconciler (v3 production)	15.47	−15.15	119,138
Chronos-bolt (univariate)	16.80	−15.12	134,586
Theta	16.91	−15.32	135,230
Seasonal-Naive(12)	17.33	−15.07	114,226
LightGBM	17.93	−13.00	169,631
TTM-r2 (multivariate)	20.61	−19.78	155,506
Unit cost			
AutoETS	**13.79**	−13.69	17,154
Ensemble [Theta, ETS, SN]	14.14	−14.82	18,134
TTM-r2 (multivariate)	16.22	−14.18	21,059
Chronos-bolt (univariate)	17.08	−15.57	21,877

- **Signed bias** is reported separately because a systematic under-forecast (negative bias) materially changes the recommended price: an engine that underestimates volume will systematically over-price.
- **RMSE** surfaces tail behaviour — a model with low MAPE but pathological monthly RMSE will still poison Monte Carlo bands.
- **Coverage** of the 90% prediction interval is tracked alongside the point metrics; we require empirical coverage $\geq 80\%$ before any forecasting model is promoted into the engine.

The *Ensemble [Theta, ETS, SN]* winner on revenue is consistent with the M-competition family of results that consistently rank simple ensembles above heavier learners on monthly business series [6]. The pretrained foundation models (Chronos-Bolt, TTM-r2) trail the ensemble on this manufacturer's data because the sample (a few dozen SKUs at sufficient density) is below the regime where foundation-model amortisation pays off; we keep them in the bake-off because we expect them to overtake the ensemble as the customer book scales.

Hierarchical reconciliation

Forecasts are reconciled across SKU → commodity-group → portfolio with the Minimum Trace (MinT-OLS) reconciler [7]. This forces additive coherence: per-SKU forecasts sum to the commodity-group forecast, and to the portfolio total. Without reconciliation, a portfolio recommendation built from unreconciled per-SKU forecasts can disagree with the portfolio-level forecast by 3–6%.

5 Win-probability model

5.1 Specification

For each (SKU, customer-segment) cluster, we fit a Bayesian logistic regression of quote outcome against log-price, controlling for tier, season, and a small number of segment fixed effects:

$$\mathbb{P}(\text{win} \mid p, x) \;=\; \sigma\big(\beta_0 + \beta_1 \log p + \beta_2 \text{tier} + \beta_3 \text{quarter} + x^{\top}\gamma\big). \tag{6}$$

Priors are weakly informative ($\mathcal{N}(0, 5)$ on coefficients, $\mathcal{N}(0, 2)$ on β_1 to express the prior belief that demand slopes negative in price). Posteriors are sampled via No-U-Turn HMC; we report the posterior median curve and a 90% credible interval band.

5.2 Low-data guardrail

Clusters with fewer than 12 quote outcomes in the trailing 540-day window are flagged *locked*: the curve still renders for visual context but the recommendation falls back to a cost-floor + market-anchor heuristic. This rule mirrors the existing production guard in the v0 engine and is documented in the workbench UI.

5.3 Conformal coverage

We post-hoc calibrate the 90% credible band with split conformal prediction [8] so that the published band has guaranteed marginal coverage of $\geq 90\%$ under exchangeability, regardless of prior misspecification.

6 Churn-response model

6.1 Two-stage decomposition

We decompose churn into a baseline component (independent of price) and a price-shock component:

$$P_{\text{churn}}(p \mid c) \;=\; \alpha(c) + \big(1 - \alpha(c)\big)\,\eta\big(\Delta p(c, p)\big), \tag{7}$$

where

- $\alpha(c)$ is the customer's baseline 12-month churn probability, sourced from Pryzm's customer-risk model (the $p_{\text{churn}}^{1q,2q,4q}$ columns in `churn_predictions.csv`, projected to 12-month horizon),
- $\Delta p(c, p) = (p - p_{\text{cur}}(c))/p_{\text{cur}}(c)$ is the relative price shock for this customer,
- $\eta(\cdot)$ is a monotone non-negative shock function fitted from historical price-change-to-renewal transitions, with shrinkage toward zero where data is thin.

This formulation guarantees $0 \leq P_{\text{churn}} \leq 1$ and recovers the baseline churn when $\Delta p \to 0$. Where insufficient price-change events exist for a cluster, η is shrunk to a global cross-portfolio estimate via empirical Bayes.

6.2 Why decomposition

A single end-to-end churn-given-price model conflates two effects we need to separate for explainability: a customer was at risk regardless of pricing (a strategic-account issue), versus a customer is at risk *because* of the proposed price (a pricing issue). The two require different mitigations and our memo surfaces them separately.

7 Demand model

The 12-month volume forecast $\mathbb{E}[V_{12} \mid p, \mathbf{sku}, c]$ combines the forecasting backbone (§4) with a price-elasticity adjustment:

$$\mathbb{E}[V_{12} \mid p, \mathbf{sku}, c] \;=\; V_{12}^{\text{forecast}}(\mathbf{sku}, c) \cdot \left(\frac{p}{p_{\text{cur}}(c)}\right)^{\epsilon(\mathbf{sku})} \tag{8}$$

where $\epsilon(\mathbf{sku})$ is a per-SKU price elasticity estimated from historical price-volume covariation when data permits, and shrunk to the commodity-group mean otherwise. We bound $\epsilon \in [-3, 0]$ to rule out positively-sloped demand.

8 Cost model

The 12-month forecast unit cost $k_{12}(\mathbf{sku})$ is drawn from the cost vintage produced by the forecasting backbone (Table 1, lower panel; current winner AutoETS, MAPE 13.79%). A safety margin τ_k (default 3% of k_{12}) is added to the cost floor in the optimisation constraint to absorb forecast bias; the margin is itself recalibrated each quarter to match observed bias.

9 Lifetime value and aggregation

The lost contribution from churn is the discounted 24-month expected margin if the customer is retained, evaluated at the current price:

$$L(c, \text{sku}) \;=\; \sum_{m=1}^{24} \frac{V_m^{\text{forecast}}(c, \text{sku}) \cdot (p_{\text{cur}}(c) - k_m(\text{sku}))}{(1+r)^{m/12}}, \tag{9}$$

with a default annual discount rate $r = 8\%$. The horizon is configurable; 24 months is the current default because it matches the typical renewal cadence of the target portfolio.

10 Constrained optimisation

10.1 Search

For a single SKU, $S(p)$ is evaluated on a price grid $\mathcal{P}$ of 41 logarithmically spaced points spanning $[0.5\,p_{\text{cur}},\ 1.8\,p_{\text{cur}}]$, followed by a golden-section refinement on the bracket containing the grid maximum. Constraints in Eq. 3 are applied as a mask over $\mathcal{P}$; any constraint that excludes the grid maximum surfaces as a *constraint-active* flag in the recommendation packet.

10.2 Solver choice

We deliberately do not use a general-purpose MIP/MINLP solver. At the per-SKU scale, the search space is one-dimensional and the objective is sufficiently smooth that a grid + golden-section search returns the global optimum in < 5 ms per SKU. We benchmarked against CVXPY (with a convex relaxation) and Pyomo (with IPOPT); neither yielded a measurable accuracy improvement and both added a deployment dependency [11, 12]. We will revisit when we expand to portfolio-level cross-SKU optimisation.

11 Monte Carlo uncertainty

11.1 Why Monte Carlo

Each input to Eq. 1 carries its own uncertainty. A point recommendation that ignores input variance routinely misstates the score by 10–40% on industrial portfolios. Monte Carlo is the standard mechanism for propagating input distributions to the score and quantifying $\mathbb{P}(S(p^\star) > 0)$ [4, 5].

11.2 Procedure

For each (SKU, p) pair we draw $N = 5{,}000$ samples and recompute $S(p)$:

1. Sample $\beta \sim$ posterior of the win-probability logistic.
2. Sample V_{12}, k_{12} from the forecasting backbone's prediction-interval distribution (split-conformal calibrated).
3. Sample $\alpha(c)$ from the customer-risk model's posterior; sample $\eta(\Delta p)$ from its empirical-Bayes posterior.
4. Recompute $S(p)$.

The reported recommendation carries

- The point estimate $\hat{S}(p^\star)$ (posterior mean),
- A 90% credible interval $[Q_{0.05}, Q_{0.95}]$ on $S(p^\star)$,
- The probability of profitability $\mathbb{P}(S(p^\star) > 0)$.

11.3 Acceptance gate

A recommendation is publishable only if $\mathbb{P}(S(p^\star) > 0) \geq 0.80$. Below that threshold the engine returns a *review-required* state with the constraint that is most often binding across the MC draws.

12 Validation methodology

12.1 Walk-forward protocol

For the 2025 backtest, the engine is shown only data dated $\leq$ 2024-12-31 at training time. For each SKU with at least one quote in 2025 it generates a recommendation $p^\star_{2025}$. The engine's recommendation is then scored against the actual 2025 invoiced behaviour. The four headline metrics are:

Table 2: Engine-level acceptance gates.

Metric	Definition	Gate
Median per-SKU margin lift	$\text{median}_{\text{sku}}[\, S(p^\star) - S(p_{\text{actual},2025}) \,]$	$\geq +3\%$
90% CI coverage	Empirical fraction of SKUs whose realised score lies inside the MC 90% CI	$\geq 80\%$
Negative-net protection	Share of SKUs with realised net_recovery < 0 at $p^\star$	$\leq 5\%$
Portfolio churn delta	$\lvert \mathbb{E}[\text{churn} \mid p^\star] - \mathbb{E}[\text{churn} \mid p_{\text{actual}}] \rvert$	≤ 2 pp
Per-SKU runtime	Wall-clock time per SKU including MC at 5,000 draws	< 500 ms

If any gate fails, the engine is held back; the notebook publishes a diagnostic report identifying the input model and failure mode.

12.2 Why these gates

- *+3% median lift* is consistent with peer-reviewed industrial-pricing case studies; lifts above +5% in B2B portfolios are anomalous and usually attributable to data drift rather than model skill [1].
- *80% CI coverage* is the lower bound for honest uncertainty: a 90% interval covering less than 80% of outcomes is empirically uncalibrated and we re-fit.
- *5% negative-net protection* is the bright-line that distinguishes a recommendation engine from a margin maximiser. A real recommendation never knowingly proposes a price with majority-negative expected contribution.
- *2 pp churn delta* matches the "trust-and-fairness" guardrail recommended in the 2025 dynamic-pricing literature [2].

13 Simulation and pre-AB modules

13.1 What-if simulator

The simulator accepts a proposed variant price and returns Low/Mid/High scenarios for 12-month revenue delta, contribution delta, and churn-risk delta against the held control. It is the same scorer (Eq. 1–2) evaluated three times under three input regimes:

- *Mid:* posterior median of every input.
- *Low:* 5th percentile of win-probability, 95th of churn-shock.
- *High:* 95th of win-probability, 5th of churn-shock.

The simulator is read-only: no quote, proposal, or audit record is written by this view. It returns to the UI in < 150 ms per call.

13.2 Monte Carlo what-if

For deeper analyses the same MC procedure described above is exposed through the simulator, returning the full $S(p)$ posterior distribution rather than three quantile slices.

13.3 Thompson sampling (post-AB)

Once an A/B test has accumulated ≥ 3 months of variant outcomes per cluster, the system can elect to switch from static-recommendation mode into a contextual-bandit policy. We use Thompson sampling with linear payoff [9] on a per-cluster basis, with the contextual features being the same inputs to Eq. 1. The exploration–exploitation tradeoff is governed by the posterior variance, which is itself the same MC machinery already in production. Published case studies report cash-margin lifts of up to $+55\%$ over a four-month deployment on an Italian e-commerce platform [10]; we treat that as an upper-bound aspiration, not a forecast.

14 Software architecture

14.1 Notebook-first methodology

The full engine is first implemented as a Jupyter notebook (`notebooks/pricing_engine_v1/`) that imports a small set of pure-Python modules (`lib/win_prob.py`, `lib/churn_response.py`, `lib/scorer.py`, `lib/optimizer.py`, `lib/monte_carlo.py`, `lib/backtest.py`). The notebook runs the 2025 walk-forward backtest and emits the acceptance-gate report. Only after the gates in Table 2 pass is the validated module promoted into the production service.

14.2 Production wiring

Promotion replaces the legacy `backend/services/pricing/recommendation.py _pick_with_floor` (which scores $(p-k)\cdot p_{\text{win}}$ only) with a thin adapter that calls into the validated engine. The FastAPI surface (`/pricing/recommendations`, `/pricing/simulate`, the workbench BFF) does not change. A new endpoint `/pricing/score_curve` exposes the full $\{p : S(p)\}$ map and the MC band so that the Pricing Studio UI can render a *score curve* with breakeven shading in place of the current win-probability curve.

14.3 Lineage and audit

Every recommendation embeds a `lineage_ref` pointing to a content-addressed manifest of:

- The forecasting vintage IDs used (volume, cost),
- The win-probability fit identifier and the cluster definition,
- The customer-risk model version and the as-of date for $\alpha(c)$,
- The Monte Carlo seed and draw count,
- The constraint values active at decision time.

This makes every accepted price reproducible from the audit log alone, which is the auditability requirement that distinguishes Pryzm from a black-box neural pricer.

15 Comparison with prior art

16 Roadmap

16.1 v1 (notebook validation)

Single-SKU, single-customer, all inputs sourced from existing services. Passes acceptance gates in Table 2 or returns a diagnostic report. Target: Q2 2026.

Table 3: Methodological comparison with the leading B2B pricing platforms. Pryzm's wedge is explicit objective + auditable lineage + uncertainty-first decisions on a notebook-validated foundation.

System	Objective surfaced	Uncertainty	Audit trail
Pricefx PricingAI [13]	Margin / revenue lift, opaque weighting	Confidence labels	Per-decision approval log
Zilliant [14]	Margin uplift via ML segment elasticity	Confidence labels	Per-deal scoring history
Vendavo [15]	Profit / win-rate target	Confidence labels	Approval workflow
Conga Price Opt. [16]	Margin / revenue targets	Limited	Approval workflow
Pryzm	**Explicit Eq. 1, decomposable inputs**	**Posterior MC band per recommendation**	**Content-addressed lineage per decision**

16.2 v2 (production wiring)

Promotion of the validated module into FastAPI; replacement of the legacy DB2-only scorer; UI surfacing of score curve, breakeven, and MC band. Target: Q3 2026.

16.3 v3 (portfolio + bandit)

Cross-SKU portfolio optimisation with substitution effects; Thompson-sampling exploration policy behind a feature flag; competitor-price signal ingestion as a real data source. Target: Q4 2026.

Acknowledgements

We thank the M-competition community, the AutoGluon and Chronos teams at Amazon, and the IBM Research Tiny Time Mixer authors for releasing their models and benchmarks. The methodology in this paper draws on the operating literature of B2B pricing, conformal prediction, and contextual bandits, and on a decade of industrial-pricing case studies; explicit credits appear in the references.

References

[1] Omnibound AI. *B2B Pricing (2026 Guide): Strategy, Models, Optimisation, and Real-World Examples That Protect Your Margin.* 2026. https://www.omnibound.ai/blog/b2b-pricing.

[2] Influencers Time. *Dynamic Pricing in 2025: Balancing Revenue and Trust.* 2025. https://www.influencers-time.com/dynamic-pricing-in-2025-balancing-revenue-and-trust/.

[3] Influencers Time. *AI Dynamic Pricing for Long-Term LTV Optimization in 2025.* 2025. https://www.influencers-time.com/ai-dynamic-pricing-for-long-term-ltv-optimization-in-2025/.

[4] Monetizely. *How to Use Monte Carlo Simulation for Pricing Risk Assessment.* 2025. https://www.getmonetizely.com/articles/how-to-use-monte-carlo-simulation-for-saas-pricing-risk-assessment.

[5] FasterCapital. *Monte Carlo Simulation and Optimization — Pricing Strategies: Insights from Monte Carlo Models.* 2024. https://fastercapital.com/content/Monte-Carlo-simulation-and-optimization--Pricing-Strategies-for-New-Ventures--Insights-f
html.

[6] S. Makridakis, E. Spiliotis, V. Assimakopoulos. *The M4 Competition: 100,000 time series and 61 forecasting methods.* International Journal of Forecasting, 36(1), 2020.

[7] S. L. Wickramasuriya, G. Athanasopoulos, R. J. Hyndman. *Optimal forecast reconciliation for hierarchical and grouped time series through trace minimization.* Journal of the American Statistical Association, 114(526), 2019.

[8] A. Angelopoulos, S. Bates. *A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification.* arXiv:2107.07511, 2021.

[9] S. Agrawal, N. Goyal. *Thompson Sampling for Contextual Bandits with Linear Payoffs.* ICML, 2013. https://arxiv.org/pdf/1209.3352.

[10] A. Aziz et al. *Personalized Dynamic Pricing Based on Improved Thompson Sampling.* Mathematics, MDPI, 2024. https://www.mdpi.com/2227-7390/12/8/1123.

[11] S. Diamond, S. Boyd. *CVXPY: A Python-embedded modelling language for convex optimisation.* Journal of Machine Learning Research, 17(83), 2016.

[12] W. E. Hart et al. *Pyomo — Optimization Modeling in Python (2nd edition).* Springer, 2017.

[13] Pricefx. *PricingAI.* https://www.pricefx.com/software/pricingai.

[14] Zilliant. *Products.* https://zilliant.com/products.

[15] Vendavo. *Our Products.* https://www.vendavo.com/our-products/.

[16] Conga. *Price Optimization and Management.* https://conga.com/platform/price-optimization-management.

A Notation

sku, c	A stock-keeping unit and a customer in its eligible book.
p, $p^\star$, p_{cur}, $p_{\text{breakeven}}$	Candidate, optimal, current, breakeven price.
$P_{\text{win}}(p)$, $P_{\text{retain}}(p)$, $P_{\text{churn}}(p)$	Per-customer probabilities, all functions of price.
V_{12}, V_m	12-month and per-month expected volume forecasts.
k_{12}, k_m, τ_k	Forecast unit cost and a safety margin for the cost floor.
$L(c, \mathrm{sku})$	24-month discounted lifetime margin (lost on churn).
$\Delta_{\max}$, κ	Contractual cap on $\lvert\Delta p\rvert$ and portfolio churn-rate cap.
$S(p)$	Cluster-level expected contribution as a function of price.
$\epsilon(\mathrm{sku})$	Per-SKU price elasticity, bounded in $[-3, 0]$.
r	Annual discount rate (default 8%).
N	Monte Carlo draw count (default 5,000).

B Change log

- **v1.0 — 2026-05-19.** Initial release. Includes 2025 walk-forward backtest of the forecasting backbone (Table 1); engine-level acceptance gates (Table 2); architecture diagram (Fig. 1); model comparison (Table 3). Subsequent revisions to update once the engine notebook completes the 2025 backtest and the 2026 forward run.